

# PDF2Data AI – Complete Project Documentation

## Table of Contents

1. Executive Summary
  2. Project Overview
  3. System Architecture
  4. Technology Stack
  5. Common Pitfalls & Solutions
- 

## 1. Executive Summary

### Project Purpose

PDF2Data AI is an enterprise-grade document processing platform that transforms unstructured PDFs into structured, application-ready data using artificial intelligence. This documentation serves as a comprehensive guide for interns joining the development team, covering everything from system architecture to implementation details.

### Key Stakeholders

- **End Users** – Organizations dealing with high-volume PDF data extraction (education, healthcare, finance, e-commerce).
- **Development Team** – Backend (Java/Spring), Frontend (React), AI/ML engineers.

### Project Goals

1. Automate data extraction from various PDF types (text, scanned, image-based).
  2. Achieve >85% accuracy in structured data extraction.
  3. Support multiple export formats (JSON, CSV, Excel, SQL, MongoDB).
  4. Process 1 000+ pages per hour.
  5. Provide a user-friendly dashboard for monitoring and management.
- 

## 2. Project Overview

### 2.1 Problem Statement

**Current Challenges in PDF Data Processing:**

- Manual data entry is time-consuming and error-prone.
- PDFs are designed for human viewing, not machine processing.
- Different PDF layouts require different extraction approaches.
- Scanned documents require OCR processing.
- Data validation and cleaning is repetitive work.

**Cost Impact:**

- Average time to manually process 100 PDFs: 40–50 hours.
- Error rate in manual data entry: 5–10%.

**2.2 Solution Overview**

PDF2Data AI addresses these challenges through:

1. **Automated Document Understanding** – AI detects document structure intelligently.
2. **Smart Data Extraction** – Extracts meaningful fields rather than raw text.
3. **Built-in Validation** – Ensures data quality and accuracy.
4. **Multiple Export Options** – Flexibility in output formats.
5. **Scalable Processing** – Handles large volumes efficiently.

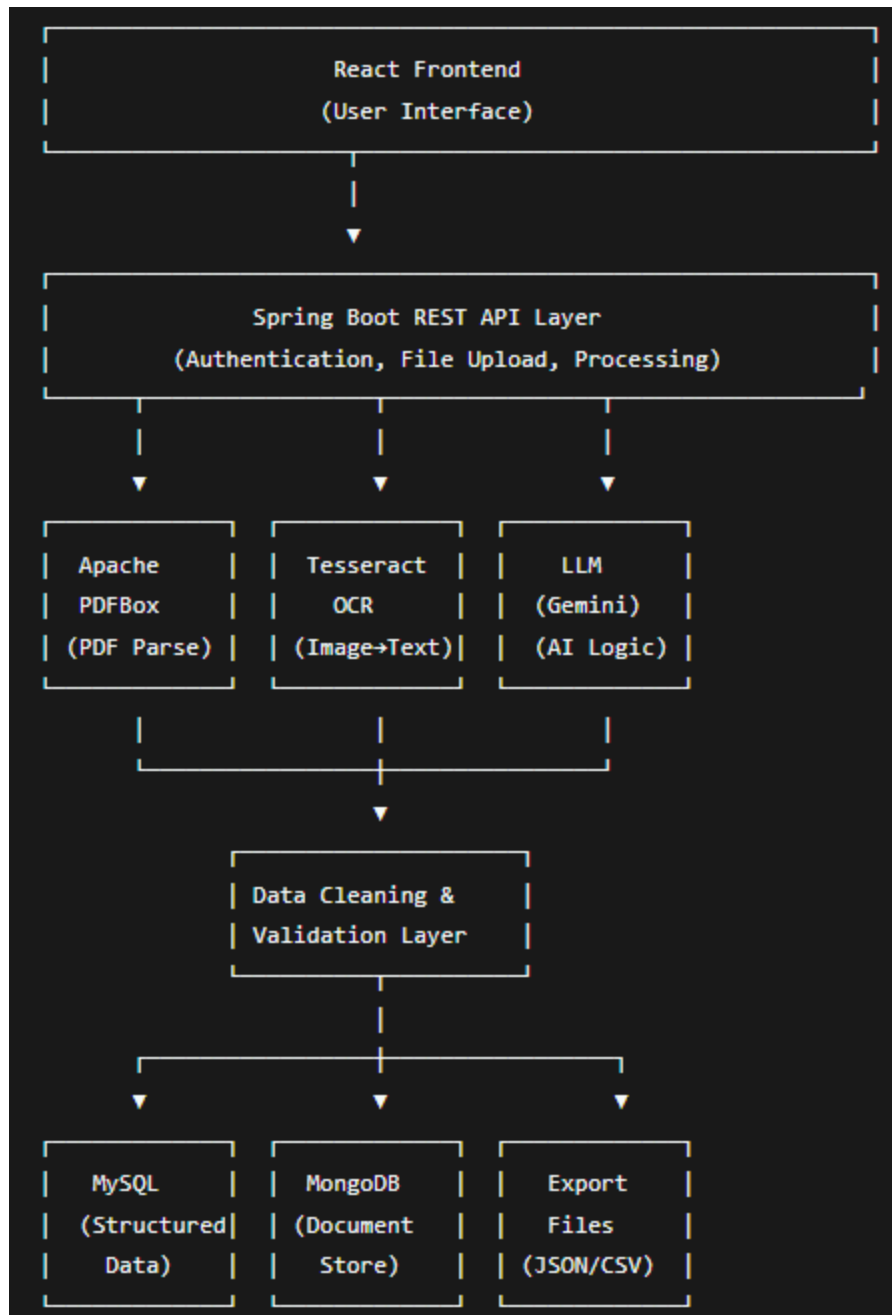
**2.3 Key Features**

| Feature                     | Description                            | Benefit                          |
|-----------------------------|----------------------------------------|----------------------------------|
| Smart Table Detection       | Detects tables without borders         | Handles complex document layouts |
| OCR Integration             | Extracts text from scanned images      | Processes all PDF types          |
| Multi-page Record Detection | Identifies record boundaries           | Accurate bulk processing         |
| Auto-schema Generation      | Creates database schemas automatically | Reduces manual setup time        |
| Field Mapping               | Recognises equivalent field names      | Improves extraction accuracy     |

---

**3. System Architecture**

## 3.1 High-Level Architecture



## 3.2 Component Details

### 3.2.1 Frontend Layer

- **Technology:** React.js with React Router.
- **Responsibilities:**
  - File upload interface (drag-and-drop).
  - Real-time processing status updates.
  - Data preview and validation interface.
  - Export options selection.

- Admin dashboard for monitoring.
- **Key Features:**
  - Responsive design.
  - Bulk upload support.
  - Progress tracking with visual indicators.
  - Error display with actionable feedback.

### 3.2.2 API Layer (Backend)

- **Technology:** Spring Boot 21 with Spring Security.
- **Key Controllers:**
  - `FileUploadController` – handles PDF uploads.
  - `ProcessingController` – manages document processing.
  - `ExportController` – handles data export.
  - `AdminController` – system monitoring endpoints.
- **Security:** JWT-based authentication with role-based access control.

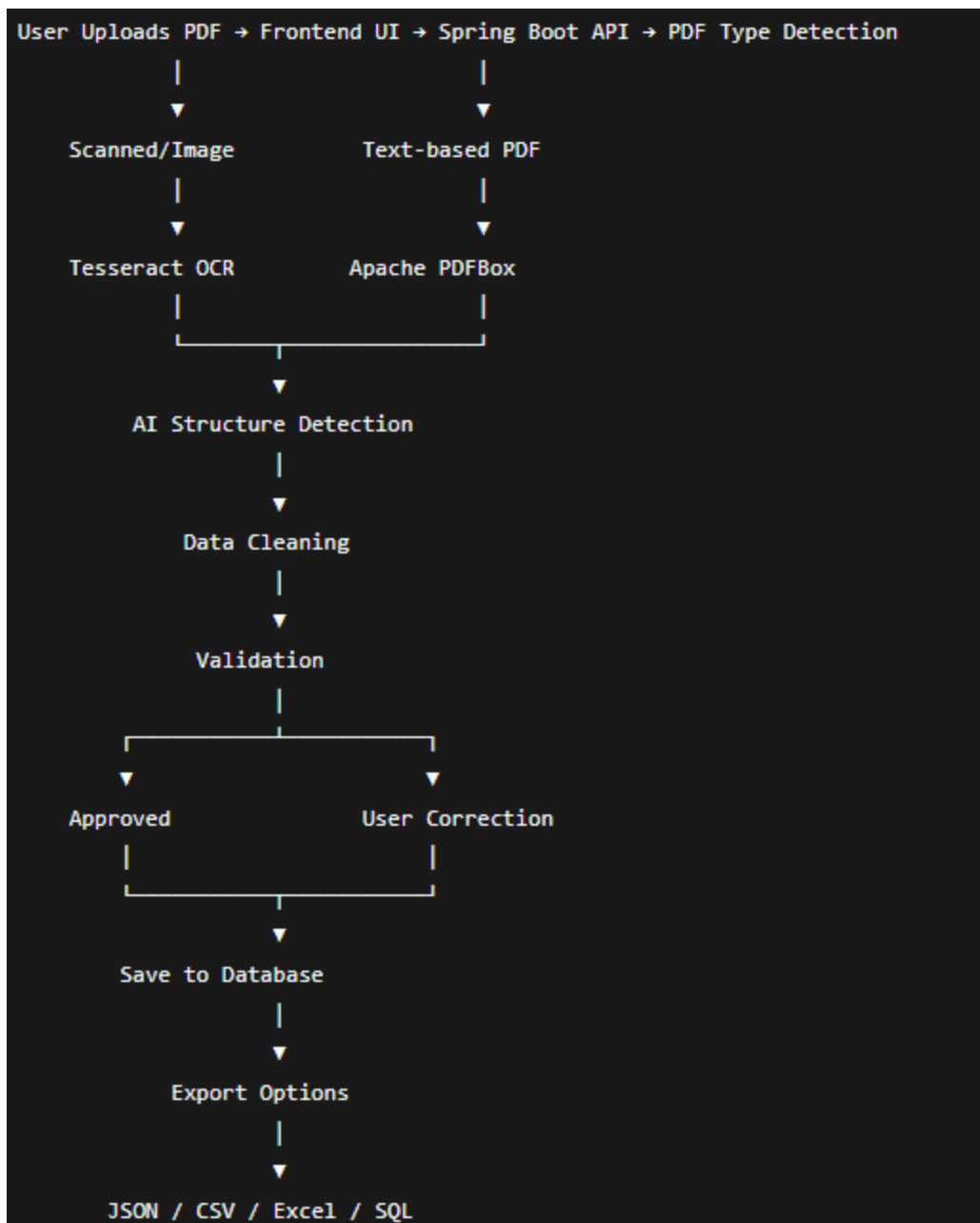
### 3.2.3 AI Processing Pipeline

text

Input PDF → Layout Analysis → Text Extraction → AI Structure Detection → Data Validation → Export

- 1. Layout Analysis** – Identifies document structure (tables, forms, paragraphs); detects page boundaries and sections; handles both text and image-based PDFs.
- 2. Text Extraction** – Uses Apache PDFBox for text-based PDFs; uses Tesseract OCR for scanned/image-based PDFs; preserves formatting and layout information.
- 3. AI Structure Detection** – Uses Gemini/OpenAI to understand document context; identifies key-value pairs, tables, and lists; maps extracted data to standardized fields.
- 4. Data Cleaning & Validation** – Removes noise (extra spaces, special characters); standardises date/time formats; validates emails, phone numbers, and other fields; handles missing or incomplete data.
- 5. Export & Storage** – Multiple export formats (JSON, CSV, Excel, SQL); direct database insertion (MySQL, MongoDB); supports both on-demand and scheduled exports.

## 3.3 Data Flow Diagram



## 4. Technology Stack

### 4.1 Backend Technologies

| Technology      | Purpose                        | Key Notes               |
|-----------------|--------------------------------|-------------------------|
| Java            | Core programming language      | Use LTS version         |
| Spring Boot     | Application framework          | Use Spring Initializr   |
| Spring Security | Authentication & Authorization | JWT-based               |
| Spring Data JPA | Database ORM                   | Hibernate behind scenes |
| Spring AI       | LLM integration                | OpenAI/Gemini support   |
| Apache PDFBox   | PDF processing                 | Handles text extraction |

| Technology    | Purpose               | Key Notes                  |
|---------------|-----------------------|----------------------------|
| Tesseract OCR | Image text extraction | Use with Tess4J wrapper    |
| MySQL         | Relational database   | For structured data        |
| MongoDB       | Document database     | Optional for flexible data |
| RabbitMQ      | Message queue         | For async processing       |
| Redis         | Caching               | Session management         |

## 4.2 Frontend Technologies

| Technology        | Version | Purpose             |
|-------------------|---------|---------------------|
| React             | 18.x    | UI framework        |
| Redux/Context API | Latest  | State management    |
| React Router      | 6.x     | Navigation          |
| Axios             | Latest  | HTTP requests       |
| React Dropzone    | Latest  | File upload         |
| Tailwind CSS      | Latest  | Styling             |
| Chart.js          | Latest  | Analytics dashboard |

## 4.3 Development Tools

| Tool          | Purpose              |
|---------------|----------------------|
| IntelliJ IDEA | Java development     |
| VS Code       | Frontend development |
| Docker        | Containerisation     |
| Kubernetes    | Orchestration        |
| Postman       | API testing          |
| Git           | Version control      |
| JUnit 5       | Testing framework    |
| Mockito       | Mock testing         |

## 5. Common Pitfalls & Solutions

| Issue                   | Root Cause                     | Solution                             |
|-------------------------|--------------------------------|--------------------------------------|
| Missing text in PDF     | PDF contains images/scans      | Enable OCR processing                |
| Garbled text            | Wrong encoding                 | Auto-detect encoding or use UTF-8    |
| Tables not detected     | Missing borders or alignment   | Enhance table detection algorithm    |
| Slow processing         | Large files or complex layouts | Use async processing with queue      |
| Memory errors           | Processing many large files    | Implement pagination and streaming   |
| Wrong field mapping     | Similar field names            | Add validation rules and synonyms    |
| Missing fields          | Incomplete prompt              | Improve prompt engineering           |
| Inconsistent formatting | Different document styles      | Train on multiple document types     |
| Hallucinations          | LLM uncertainty                | Add confidence scores and validation |